

Lab 10: Indexing and Query Plans

Dheeraj Kumar
23B0920

April 2026

1 Part - 1

Task 1.1

Execution time = 23.184 ms

Task 1.2

Query Plan:

Seq Scan on ipl_deliveries (cost=0.00..170.00 rows=1 width=8)

Filter: (ball_id = 1000)

Execution time = 0.755 ms

Scan Type: Seq Scan (Sequential Scan)

It is slow because it looks at every row of the table

Task 1.4

Query Plan:

Index Scan using idx_ball_id on ipl_deliveries (cost=0.29..8.30 rows=1 width=8)

Index Cond: (ball_id = 1000)

Execution time = 0.056 ms

Scan Type: Index Scan

It is faster because building B-tree allows it to search in $O(\log n)$ instead of $O(n)$

Task 1.6

Execution time = 30.164 ms

It is slower than insertion without Index

It is slower because without index postgres doesn't need to do any extra work for insertion, $O(1)$, but with indexing, it needs to maintain the B-tree structure, so $O(\log n)$

2 Part - 2

Task 2.1

Query: EXPLAIN SELECT * FROM takes WHERE grade = 'A';

Output:

QUERY PLAN

Seq Scan on takes (cost=0.00..595.00 rows=1 width=24)

Filter: ((grade)::text = 'A'::text)

Postgres chose Seq scan because it doesn't have any index for searching

It chooses Seq scan over Index scan (when index is there) if

$\text{cost}(\text{seq scan}) < \text{cost}(\text{index scan} + \text{heap fetches})$

Task 2.2

Query:

EXPLAIN SELECT * FROM student WHERE dept_name = 'History' AND id = '1968';

Output:

QUERY PLAN

Index Scan using student_pkey on student (cost=0.28..8.30 rows=1 width=24)

Index Cond: ((id)::text = '1968'::text)

Filter: ((dept_name)::text = 'History'::text)

Since id is the primary key of student, the predicate id = '1968' triggered index scan but index isn't built for dept_name hence it applied filter on previous output

Task 2.3

Index creation: CREATE INDEX idx_takes_course_id ON takes(course_id);

Query:

EXPLAIN SELECT * FROM takes WHERE course_id = '401';

Output:

QUERY PLAN

Bitmap Heap Scan on takes (cost=6.57..241.97 rows=295 width=24)

Recheck Cond: ((course_id)::text = '401'::text)

– > Bitmap Index Scan on idx_takes_course_id (cost=0.00..6.50 rows=295 width=0)

Index Cond: ((course_id)::text = '401'::text)

Bitmap Index Scan: It scans index and builds a bitmap of matching row locations and then sort/organize them and then fetches rows in bulk from heap

Postgres used Bitmap scan because it has an index over course_id and many rows match the filter

Task 2.4

Query:

```
EXPLAIN SELECT * FROM course c JOIN takes t ON c.course_id = t.course_id
WHERE c.course_id = '401';
```

Output:

```
QUERY PLAN
-----
Nested Loop (cost=6.57..249.42 rows=295 width=59)
-¿ Seq Scan on course c (cost=0.00..4.50 rows=1 width=35)
  Filter: ((course_id)::text = '401'::text)
-¿ Bitmap Heap Scan on takes t (cost=6.57..241.97 rows=295 width=24)
  Recheck Cond: ((course_id)::text = '401'::text)
-¿ Bitmap Index Scan on idx_takes_course_id (cost=0.00..6.50 rows=295 width=0)
  Index Cond: ((course_id)::text = '401'::text)
```

Outer: course (first child)

Inner: takes (second child, uses bitmap index)

This is efficient because Outer produces very few rows (filtered by Primary key) For each outer row it does index lookup in inner table

If inner table(takes) has no index then, For each outer row it does full scan of takes

Task 2.5

QUERY PLAN

```
-----
Insert on takes2 (cost=0.00..520.00 rows=0 width=0) (actual time=25.729..25.730
rows=0 loops=1)
-¿ Seq Scan on takes (cost=0.00..520.00 rows=30000 width=24) (actual time=0.010..2.743
rows=30000 loops=1)
Planning Time: 0.084 ms
Execution Time: 25.755 ms
```

Task 2.6

QUERY PLAN

Insert on takes2 (cost=0.00..520.00 rows=0 width=0) (actual time=67.427..67.428 rows=0 loops=1)
 -ĩ Seq Scan on takes (cost=0.00..520.00 rows=30000 width=24) (actual time=0.013..1.480 rows=30000 loops=1)
 Planning Time: 0.124 ms
 Execution Time: 67.451 ms

Primary key introduces Index maintenance(B-tree insert per row) and Uniqueness check, hence it is slower than without primary key
 The overhead is justified because PK ensures Data integrity (no duplicates),
 Faster joins and lookups

Task 2.7

QUERY PLAN

Aggregate (cost=68.84..68.85 rows=1 width=8)
 -ĩ Nested Loop Semi Join (cost=0.29..68.63 rows=85 width=0)
 -ĩ Seq Scan on course c (cost=0.00..4.00 rows=200 width=4)
 -ĩ Index Only Scan using idx_takes_course_id on takes t (cost=0.29..7.03 rows=353 width=4)
 Index Cond: (course_id = (c.course_id)::text)

Task 2.8

QUERY PLAN

Aggregate (cost=68.84..68.85 rows=1 width=8)
 -ĩ Nested Loop Semi Join (cost=0.29..68.63 rows=85 width=0)
 -ĩ Seq Scan on course c (cost=0.00..4.00 rows=200 width=4)
 -ĩ Index Only Scan using idx_takes_course_id on takes t (cost=0.29..7.03 rows=353 width=4)
 Index Cond: (course_id = (c.course_id)::text)

This is exactly same as last query output, postgres doesn't differentiate between Exists and in